

PRISMA

Enterprise Multi-Industry Predictive Analytics
& Supply Chain SaaS

Complete Project Overview — Features, Architecture & Production Readiness

Verticals: Apparel & Fashion · Consumer Electronics · Pharmaceuticals (GxP / 21 CFR Part 11)

Generated 2026-06-27 | Repository: Desktop/Prisma | Status: Pre-production hardening

1. Executive Summary

PRISMA is a vertically-integrated, multi-tenant SaaS platform that combines demand forecasting with supply-chain optimization, purpose-built for three industries: **Apparel & Fashion**, **Consumer Electronics**, and **Pharmaceuticals** (with GxP / 21 CFR Part 11 controls), plus emerging Agriculture/Agrocenter and B2B Hardware verticals. The platform pairs a production-grade machine-learning forecasting stack (11 models across 4 families, with a zero-shot foundation-model fallback) with a FastAPI backend enforcing strict PostgreSQL row-level-security multi-tenancy, and a React/TypeScript dashboard with industry-specific views.

The codebase is substantial and was built in five tracked phases (marketing site → schema/backend → ML stack → frontend → production hardening), totalling roughly 184+ tracked deliverable files. The engineering culture is notably rigorous: several performance and architecture decisions in this codebase are documented as *measured and rejected* rather than assumed — e.g. a tempting single-query rewrite of the sales summary endpoint was benchmarked and found 1.8x **slower**, so the original design was kept deliberately, with the rationale recorded inline so it isn't "re-optimized" back into a regression.

At a glance

Dimension	Current state
Architecture	3 services (React frontend, FastAPI backend, FastAPI ML inference API) + PostgreSQL 16 (pgvector, RLS) + Redis
Multi-tenancy	PostgreSQL FORCE ROW LEVEL SECURITY, fail-closed startup guard against superuser/BYPASSRLS roles
ML stack	11 models / 4 families (foundation, deep, GBT, statistical) + stacked ensemble + Chronos zero-shot fallback
Integrations	34-provider catalog; Shopify, CSV/Excel upload, REST push ingest are genuinely live; rest are connect-and-queue
Deployment	Docker Compose (dev) and Kubernetes via Kustomize base + staging/production overlays
CI/CD	6 GitHub Actions workflows: backend/frontend/ml CI, image build + Trivy scan, staging auto-deploy, gated prod prod
Security posture	Strong tenant isolation; RBAC, MFA, SSO/SCIM, and tamper-evident audit are still gaps (see §6)
Compliance target	GDPR, SOC 2, ISO 27001, FDA 21 CFR Part 11 — all assessed as partial/in-progress, none ce

2. System Architecture

PRISMA runs as three independently deployable services in front of a shared PostgreSQL + Redis data tier:

```
Ingress (nginx, TLS) ■■ / → frontend (React+nginx, :8080)
■■ /api/ → backend (FastAPI, :8000) ■■ auth, industry router, GxP batches, RLS-scoped DB
■■ /ml/ → ml-api (FastAPI, :8001) ■■ Chronos zero-shot + trained ensemble (joblib)
both services share → PostgreSQL 16 (+pgvector, +RLS) and Redis
```

Multi-tenancy model

- Every domain table carries a **tenant_id** UUID; isolation is enforced at the database layer via a PostgreSQL RLS policy (`tenant_id = current_tenant_id()`), not just application code.
- The JWT carries `tid` (tenant), `ind` (active industry), and `role`; **TenantContextMiddleware** writes these into request state for every router and DB session.
- Every async DB session issues `SET LOCAL app.current_tenant_id` inside its transaction — safe under PgBouncer transaction pooling, with no GUC leakage across pooled connections.

- A dedicated **prisma_app** Postgres role is created with `NOSUPERUSER NOBYPASSRLS`; the backend has a fail-closed startup guard that **refuses to boot in production** if the connected role could bypass RLS — this is the single strongest control in the system.
- Workspace switching: the SPA can send `x-Industry-Code` to override the JWT's default industry for a single request (useful for tenants active in multiple verticals).

Forecast request data flow

client → POST /forecast → ml-api

■ trained artifact exists? → InferenceService → load joblib bundles → run StackedEnsemble → persist forecast_results

■ no artifact → ZeroShotForecaster → pull history (RLS) → Chronos.predict() → empirical p10/p50/p90 → persist

Both paths return a `ForecastResponse` with a source discriminator so the UI can show the user whether a forecast is tenant-trained or a foundation-model zero-shot estimate.

ML training methodology (5 stages)

- **DATA** — load the tenant's sales panel from Postgres (RLS-scoped) and classify intermittency.
- **PRE-TRAIN** — the model registry instantiates the foundation/deep/GBT/statistical models configured for that industry.
- **DOMAIN FIT** — each model is trained on the tenant's own panel.
- **STACK** — a stacked ensemble solves a constrained optimization (SLSQP) to minimize pinball loss on a holdout window, producing convex blend weights — a genuine probabilistic stacker, not a naive average.
- **VALIDATE** — walk-forward backtest against a full metric suite, logged to MLflow.

3. Feature Inventory

3.1 Backend API surface (~25 routers)

Area	Routers / capability
Core & auth	auth (Firebase JWT + dev Argon2 login, session, password reset); industry_router (per-vertical context + RBAC gate)
Forecasting	forecasts (ML proxy + Chronos fallback); hybrid_forecast (trend-adjusted Pessimistic/Base/Optimistic scenarios); forecasts
Signals	trends (FRED, Google Trends, Reddit, news → TrendScore); signals (manual external-event ingestion: weather, competitor)
Supply chain	inventory (stock, EOQ, holding cost); shortage_alerts (rules-based alerting + ack/resolve workflow); products / skus (catalog)
Financial	sales_analytics (revenue by product/region/channel); financial (revenue-at-risk, excess-inventory cost); export (bulk CSV)
Integrations	integrations_hub (34-provider catalog, generic connect/upload); integrations (Shopify OAuth + live polling); webhooks (d
Cross-industry	cross_industry (correlation analysis across verticals); billing (Razorpay subscriptions, usage metering, tier enforcement)
Industry verticals	industries/pharma, fashion, electronics, agrocenter, hardware — vertical-specific endpoints (see §3.5)
Platform	demo_request (self-serve trial provisioning); websocket (live forecast/alert push via Redis pub/sub)

3.2 ML / forecasting stack (backend/ml)

- **Foundation (zero-shot):** Chronos (Amazon T5), TimesFM (Google), Moirai (Salesforce), Lag-Llama.
- **Deep learning:** DeepAR, N-HiTS, Temporal Fusion Transformer (TFT).
- **Gradient boosting:** LightGBM.
- **Statistical:** ETS, Croston (intermittent demand), ADIDA, seasonal-naive / naive baselines.
- **Ensemble/meta:** stacked ensemble (SLSQP convex weighting on pinball loss); isolation-forest anomaly detector; per-SKU meta-learner for model routing.
- **Causal & optimization:** DoWhy-based causal/uplift inference for price & promo elasticity; supply-chain optimization (EOQ, multi-echelon, replenishment, markdown, RL replenishment).
- **Differentiator — censored-demand correction:** stockout-driven zero sales are "uncensored" before training so models learn true demand rather than under-ordering on the next cycle; most competing tools skip this.
- **Serving:** REST inference API with admission-control throttling (max concurrent inferences, queue cap, per-tenant rate limit, 20s timeout) and Chronos pre-load on startup for fast first-request latency.
- **Monitoring:** drift detection (PSI/KL/KS/Wasserstein) and production backtest accuracy tracking — currently detects drift but does not yet auto-trigger retraining (tracked gap, \$7).

3.3 Frontend (React + TypeScript)

- Industry-routed dashboards: Fashion (collection drops, size/color heatmap), Electronics (SKU lifecycle, EOL forecasts), Pharma (batch/expiry/GxP timeline), Agrocenter (crop calendars, weather overlay), Hardware (B2B tiers, MOQ constraints).
- Forecasting UI: primary forecast view with quantile bands, hybrid trend-adjusted scenarios, accuracy/backtest dashboard, and a What-If scenario builder (lead-time / price / demand-shock simulation).
- Supply chain: Inventory (stock, EOQ, reorder), Shortage Alerts (rule editor + workflow), Products/SKUs catalog with bulk import/export, Pharma batch lifecycle with QA sign-off audit trail.

- Analytics & signals: Sales Analytics, Trend Analysis (FRED/Reddit/news), manual Signals tracker, Cross-Industry Correlation, Financial Impact (revenue-at-risk).
- Integrations Hub UI: full 34-provider catalog grouped by category, connect/disconnect flows, live/beta/planned status badges, CSV/Excel upload.
- Real-time: live Shopify order stream over WebSocket; webhook endpoint management/test-delivery/retry inspection.
- Tenant/admin: Settings, Billing (Razorpay usage + invoices), Profile, plus public marketing pages (landing, case studies, testimonials) and auth flows (login, forgot/reset password).

3.4 Integrations Hub

A single catalog (`backend/app/connectors/catalog.py`) lists all 34 providers with category, availability (live / beta / planned) and auth-field schema; the frontend renders directly from `GET /api/v1/integrations/catalog`, so adding a new provider entry is a one-line catalog edit.

Status	Providers
Live	CSV/Excel upload, Shopify (OAuth + live polling), WooCommerce, generic REST API push (token-auth), g
Beta	Direct read-only SQL sync: PostgreSQL, MySQL
Planned (creds captured, adapter TBD)	Magento, BigCommerce, Amazon Seller Central, Flipkart, Square, Lightspeed, Toast, Clover, SAP, NetSu

Connecting a "planned" provider stores encrypted credentials and records a connection request for later adapter work — nothing is silently dropped. Push ingest is token-authenticated: the connect step mints a token, stores only its SHA-256 hash, and returns the raw token once.

3.5 Industry-vertical features

Vertical	Specialized capability
Pharma (GxP)	Batch lifecycle (quarantine→QA review→released/rejected/recalled/expired); lot/NDC tracking; cold-chain temperature audit
Fashion	Collection-drop calendars; per size/color variant demand forecasting; seasonal intensity tracking; fashion-week / search-inte
Electronics	Component/BOM dependency tracking; end-of-life forecasting; multi-variant demand (color/capacity/generation); long-lead-
Agrocenter	Planting/harvest calendars; weather-driven demand modeling; commodity price correlation via FRED + custom signals
Hardware (B2B)	Minimum-order-quantity constraints; distributor tier analytics; regional stock pooling across warehouses

4. Production Infrastructure

4.1 Local / Docker Compose topology

docker-compose.yml defines 8 services: postgres, backend, worker, ml-api, redis, frontend, prometheus, grafana — with named volumes for Postgres data, ML artifacts, MLflow runs, the Hugging Face model cache, and metrics/dashboard state.

Service	Role
postgres	PostgreSQL 16 + pgvector + RLS policies (schema/migrations applied via Alembic)
backend	FastAPI app (:8000) — auth, all domain routers, tenant-scoped DB sessions
worker	Background singleton loops (webhook retry, pollers) — leader-elected via Redis lease when multiple replicas run
ml-api	FastAPI inference service (:8001) — trained ensemble + Chronos zero-shot fallback
redis	Rate limiting, response cache, idempotency keys, leader election, webhook replay protection
frontend	nginx + built React SPA (:8080)
prometheus / grafana	Metrics scrape (:9090) and dashboards (:3000); anonymous viewer in dev, must be locked down in prod

4.2 Kubernetes topology

Kustomize-based: a base/ manifest set (namespace, backend, frontend, ml-api, postgres, redis, ingress, PodDisruptionBudgets, migrate Job, secrets template) with **staging** and **production** overlays. The staging overlay applies a `stg-` name prefix and reduced replica counts; production overlay adjusts replicas, resource requests, and adds a dedicated DB pooler config. Scheduled CronJobs handle WAL/base backups, automated backup-restore validation, and quarterly partition maintenance for the range-partitioned `historical_sales` / `forecast_results` tables.

4.3 CI/CD pipelines (.github/workflows)

Workflow	Purpose
backend-ci.yml	Lint + type-check + pytest (incl. <code>test_rls_isolation.py</code>) + pip-audit
frontend-ci.yml	Lint, type-check, build the React app
ml-ci.yml	ML package tests
security.yml	gitleaks secret scan (full git history) on every push/PR + weekly schedule; bandit static analysis on backend & ML
build-images.yml	Builds & pushes backend/ml-api/frontend images to GHCR; Trivy vulnerability scan uploaded to the Security tab;
deploy.yml	Auto-deploys to staging after a green image build; runs DB migrations as a Job before rollout; production promotion is

Release flow: PR → main triggers backend/ml/frontend CI → on green, images build and push with branch/SHA/semver/latest tags → staging auto-deploys → manual human-gated promotion to production (GitHub Actions environment approval) → migration Job runs `alembic upgrade head` before the rollout, never after.

4.4 Observability

Signal	Source	Destination
--------	--------	-------------

Structured logs	structlog (JSON)	stdout → Loki / CloudWatch
Metrics	prometheus_client	/metrics (token-protected in prod) → Prometheus → Grafana
Traces	OTLP/gRPC (optional)	OpenTelemetry Collector
Audit trail	audit_log table, append-only via SQL RUPS	PostgreSQL
ML experiment tracking	MLflow SDK	Volume-mounted file backend

5. Reliability, Disaster Recovery & Multi-Region

5.1 Database HA

- PostgreSQL is managed by the **CloudNativePG** operator: 3 instances on required pod anti-affinity (primary + sync replica + async replica), automatic failover.
- **Synchronous replication** to at least one replica — a committed write survives loss of the primary with zero data loss.
- **Continuous WAL archiving** to S3 (barman object store) enables point-in-time recovery to any moment.
- App traffic flows through **PgBouncer** poolers (read/write split); migrations bypass the pooler and connect directly, since DDL must not run through a transaction pooler.
- **Targets:** RPO ≤ 5 minutes (continuous WAL), RTO ≤ 30 minutes — and a weekly automated restore drill actually measures the real RTO rather than assuming it.

5.2 Backup validation (automated, not assumed)

Job	Cadence	What it proves
postgres-backup-freshness-check	Daily 03:30 UTC	Newest base backup is <26h old and WAL archiving is continuous
postgres-backup-restore-drill	Weekly Sun 05:00 UTC	The backup actually restores: replays WAL, runs pg_verifybackup, asse

Failures page on two critical Prometheus alerts: `PrismaBackupValidationFailing` and `PrismaBackupStale`. PITR is performed by booting a *new* recovery cluster from object storage and explicitly cutting traffic over — it never mutates the live cluster in place.

5.3 Multi-region / data residency

PRISMA runs as independent **regional cells** — each a full, self-contained stack (postgres + redis + backend + ml-api + frontend) bound to one residency zone (us-east, us-west, eu-west, eu-central, ap-south, ap-northeast). A tenant's data lives in exactly one cell (`tenants.home_region`); cross-cell data movement is explicitly forbidden, so an EU tenant's PII/sales/signals/batches/ML artifacts never leave the EU cell — a direct GDPR data-residency control.

- A lightweight global control-plane handles marketing, region listing, and login redirection (resolves tenant → home region, 302s to the cell host).
- A `RegionRouterMiddleware` backstop returns **421 Misdirected Request** if a request reaches the wrong cell; the SPA reads the JSON body and hard-navigates since browsers don't auto-retry 421.
- Only marketing/login redirection, Razorpay billing webhooks (routed via a tenant-id note on the customer), and anonymized Prometheus federation legitimately cross cell boundaries.
- A full-cell DR rehearsal (DNS cutover + PITR restore + overlay re-apply + health validation) is performed quarterly.

6. Security & Compliance Posture

An internal security review (2026-06-15) rates PRISMA's **tenant isolation as genuinely strong and ahead of most early-stage multi-tenant SaaS** — forced RLS, a non-superuser runtime role, and a fail-closed boot guard. **No critical, actively-exploitable findings** were found in the reviewed code paths. The gaps are concentrated in enterprise identity lifecycle and authorization breadth, not in core isolation.

6.1 What's strong today

- **Row-level security:** FORCE RLS on every tenant table; policy blocks cross-tenant reads *and* inserts; `current_tenant_id()` defaults to NULL → zero rows when unset (fail-closed).
- **Role hardening:** runtime DB role is NOSUPERUSER/NOBYPASSRLS; the app refuses to start in production otherwise.
- **Authentication:** delegated to Firebase (no hand-rolled crypto); brute-force lockout per-account and per-IP via Redis.
- **Secrets:** fail-closed weak/default-secret rejection in production, plus an automated secrets-audit script run pre-deploy.
- **Resilience primitives:** idempotency keys for mutating requests, webhook replay protection (de-dupe on provider delivery ID), Redis-backed leader election for singleton background loops.

6.2 Known gaps (no critical vulnerabilities, but enterprise/compliance blockers)

ID	Area	Severity	Finding
SEC-1	Authorization	High	RBAC (<code>require_role</code>) is wired into only ~4 of ~21 routers; most mutating endpoints rely on tenant scoping
ID-2	Identity	High	MFA columns exist in schema but are never read or enforced — no TOTP/WebAuthn flow
ID-3	Identity	High	SSO/SAML not configured per tenant (Firebase→GCIP path exists but unconfigured); no SCIM 2.0 user
SEC-3	Audit	High	<code>audit_log</code> is append-only at the DB layer but not cryptographically tamper-evident, and not WORM-arch
SEC-5	Crypto	High	Integration-credential encryption uses a statically-derived key with no KMS, no rotation, and no per-ten

Full gap register and a phased remediation roadmap (Phase 0: RBAC sweep + audit hardening, weeks 0–4 ·

Phase 1: SSO/MFA, weeks 4–10 · Phase 2: SCIM + tamper-evident audit, weeks 10–18 · Phase 3: GDPR DSAR &

Part 11 e-signatures, continuous) live in `docs/SECURITY_AND_COMPLIANCE_REVIEW.md`.

6.3 Compliance readiness (technical controls only — not certification)

Framework	Status	Primary remaining gap
GDPR	Partial	Right to erasure (Art. 17) not built; DSAR export needs formalizing; no RoPA/sub-processor register
SOC 2	Partial	Change management is strong (Alembic + git + CI); logical access (MFA, RBAC breadth) is the ope
ISO 27001	Partial	Secure development practices already strong; ISMS/org policies and key-management lifecycle are
FDA 21 CFR Part 11	Partial — ph	MFA blocking, tamper-evident audit + e-signature manifestation/linking are the named critical path items

7. Performance Engineering (measured, not estimated)

A dedicated backend/database performance pass produced runnable benchmarks against real Postgres 16 + Redis 7 (backend/benchmarks/), with every number below reproducible by anyone with the repo. Two intuitive "optimizations" were tested and rejected by measurement rather than shipped on assumption.

Change	Result	Notes
Auto partition maintenance for historical sales aggregates	Eliminates aggregate results outages	Static partitions only covered 2022–2027; a daily CronJob + migration
Covering index on the sales-summary table	2x faster (29.7ms → 24.5ms)	Index-only scan after VACUUM sets the visibility map; the redundant of
Redis response cache on dashboard	100x faster; 109x faster on cache hits	Q24.5ms → 0.22ms; invalidation on write; fails open (passthrough) if Redis
Atomic Lua rate-limiter (single EVALSHA)	Serial on the happy path; removes a lock	Old NOREXPIRE+TTL sequence could leave a key with no TTL on a
Single-FILTER rewrite of the 8-query dashboard	1.8x slower	The single query must scan the full ~1.5-year window and apply 8 filters

Net effect on the analytics hot path: the first request per cache-TTL window costs roughly the (now ~1.2x faster) DB query; every subsequent poll within that window costs ~0.4ms instead of ~25ms — a ~99% reduction on the overwhelming majority of real dashboard traffic, since dashboards poll repeatedly. JWT validation, ML batching/GPU support, and frontend bundle/virtualization were assessed as already adequate or out of scope for this pass and are tracked as backlog, not implemented.

8. AI / Forecasting Strategy & Roadmap

An internal AI strategy review (2026-06-15) characterizes the ML stack as "far more mature than a greenfield" — the work ahead is sharpening and closing gaps, not building from scratch.

8.1 What was added this pass

- A **skill-vs-naive scoreboard** in the production backtest: every model now must beat a seasonal-naive baseline (`skill_vs_naive > 1`) to be considered "worth its complexity" — a guardrail against shipping fancy models that quietly underperform a simple baseline.
- A real **MASE calculation bug fix** (the seasonal-difference scale was computed as an n-th order derivative instead of a lag-season difference) in both the backtest and the production drift-monitoring code.
- A dependency-free, CI-able **offline benchmark harness** using synthetic demand archetypes (seasonal, trending, intermittent, promo-spiky) run through the real walk-forward backtest code — no live DB or heavy ML deps required.

8.2 Honest findings from the offline benchmark

On synthetic seasonal demand (40 series × 156 weeks, 4 walk-forward folds), seasonal-naive beat plain-naive by ~11% as expected — validating the measurement framework itself. A genuine calibration finding: 80%-nominal prediction intervals showed 93–95% empirical coverage, meaning the bands are measurably over-wide — flagged as a concrete, fixable accuracy opportunity (conformal calibration) rather than glossed over.

8.3 Competitive differentiators already in the codebase

- Censored-demand correction** — forecasts true demand through stockouts instead of the censored sales signal most competitors use.

- **Causal/uplift-aware forecasting** via DoWhy — promo and price effects modeled causally, not just correlationally.
- **Decision-grade probabilistic output** piped directly into newsvendor / multi-echelon / markdown optimizers, not point forecasts thrown over the wall.
- **Multi-industry transfer** — one platform, tuned stacks per vertical, with a foundation-model cold start for brand-new tenants.

8.4 Roadmap

Phase	Focus	Outcome
P0 (0–4 wk)	Conformal interval calibration; skill-gated model promotion; close	Standardized forecasts + an automated trust guardrail
P1 (1–3 mo)	Champion-challenger promotion; model governance/cards; meta	Covered self-improving model deployments
P2 (2–4 mo)	SAP + NetSuite ERP connectors; decision-regret monitoring	Enterprise data reach + dollar-framed value story
P3 (4–6 mo)	Oracle + Dynamics connectors; conformal service-level UX	Full ERP coverage + planner-facing differentiator

9. Production Release Checklist (current branch)

The repository includes a working production-hardening checklist for promoting to a live environment. Several items are explicitly **blocking**: the backend will not boot in production until they are resolved, by design.

#	Step	Blocking
0	Rotate exposed dev-tier secrets: Supabase DB password, Firebase service-account key, JWT_SECRET, ML_SERVICE_TOKEN, Razorpay	Yes
1	Create/confirm the non-privileged prisma_app DB role (NOSUPERUSER NOBYPASSRLS); point runtime DATABASE_URL at it, never at	Yes
2	Provision real Kubernetes secrets (JWT_SECRET, ML_SERVICE_TOKEN, DB URLs, Firebase credentials, backup-aws S3 keys) &mdash	Yes
3	Wire Prometheus to send the METRICS_TOKEN bearer header, or /metrics goes dark in production	Operational
4	Confirm CI is green: backend-ci (incl. RLS isolation test), frontend-ci, ml-ci, security (gitleaks+bandit), build-images (Trivy CRITICAL gate)	Gate
5	Merge to main → auto-deploy to staging → verify RLS isolation manually + a forced 403 on an under-privileged role	Verification
6	Promote to production via a manual, environment-approval-gated GitHub Actions run; migration Job runs before gate	Is before gate
7	Post-deploy: confirm /health is 200, logs show database.rls_check.ok, a cross-tenant probe returns no error, the nightly backup CronJob	Verification

9.1 Cost & scaling levers

Service	Bottleneck	Knob
backend	CPU (auth, JSON)	Horizontal Pod Autoscaler on CPU (70%)
ml-api	CPU (foundation-model inference)	HPA on CPU (75%); switch to chronos-t5-small (~300MB vs ~3GiB) in constrained environment
postgres	Connections / IO	Vertical scale-up; add a read replica and route reads via a separate engine URL
frontend	Bandwidth	Put a CDN in front of the Ingress

This document was assembled from the project's own architecture, operations, performance, security, and AI-strategy reviews (docs/), the CI/CD workflow definitions, Kubernetes/Compose manifests, and a structural read of the backend, ML, and frontend source trees. It reflects the state of the repository as of 2026-06-27.